

DATA ENGINEERING COURSE

Advanced Functions

CLASS NOTES

`*args`

`**kwargs`

`Lambda`

`map()`

`filter()`

Clean Code

Instructor: **Tanishq Tyagi**

Comprehensive study guide covering advanced Python functions for Data Engineering

Table of Contents

1. Session Overview
2. Learning Objectives
3. Why Advanced Functions?
 - 3.1 The Problem with Basic Functions
 - 3.2 Advanced Functions in Industry
4. Arbitrary Arguments
 - 4.1 *args — Multiple Positional Arguments
 - 4.2 **kwargs — Variable-length Keyword Arguments
 - 4.3 Combining *args and **kwargs
 - 4.4 *args vs **kwargs — Key Differences
5. Lambda Functions
 - 5.1 Syntax
 - 5.2 Single & Multiple Arguments
 - 5.3 When to Use Lambdas
6. The map() Function
 - 6.1 Syntax & Flow
 - 6.2 Practical Examples
7. The filter() Function
 - 7.1 Syntax & Flow
 - 7.2 Practical Examples
8. Clean Code Principles
 - 8.1 Meaningful Names
 - 8.2 Small Functions
 - 8.3 Avoid Repetition (DRY)
 - 8.4 Single Responsibility Principle
9. Practical Examples
10. Real-World Data Engineering Perspective
11. Summary & Quick Revision Sheet
12. Interview Questions — General Python
13. Data Engineer Interview Preparation

1. Session Overview

This session dives into **advanced function features in Python** that are widely used in professional and production-grade code — especially in Data Engineering pipelines. We move beyond basic function definitions to explore flexible argument handling (**args* and ***kwargs*), concise anonymous functions (*lambda*), and two of Python's most powerful higher-order functions — *map()* and *filter()*. The session concludes with an introduction to **Clean Code Principles**, which are the hallmarks of professional, maintainable software.

Together, these concepts form the foundation for writing **flexible, reusable, and efficient data transformation code** — a core skill for any Data Engineer working with ETL pipelines, batch processing jobs, or real-time data streams.

2. Learning Objectives

By the end of this session, students will be able to:

- Explain why advanced function features are preferred over basic functions in production environments.
- Define and use ***args** to write functions that accept any number of positional arguments.
- Define and use ****kwargs** to write functions that accept any number of keyword arguments.
- Combine ***args** and ****kwargs** in the same function and understand ordering rules.
- Write concise **lambda (anonymous) functions** for single-expression operations.
- Apply the **map()** function to transform every element in an iterable using a function.
- Apply the **filter()** function to selectively retain elements from an iterable.
- Distinguish between `map()` and `filter()` and know when to use each.
- Write code that follows **Clean Code Principles**: meaningful names, small functions, DRY, and single responsibility.
- Connect all concepts to real-world **Data Engineering use cases** including ETL, data pipelines, and production systems.

3. Why Advanced Functions?

3.1 The Problem with Basic Functions

In beginner Python, we define functions with a fixed set of parameters. This works for simple scripts but becomes limiting quickly. Consider a function that adds two numbers — what if tomorrow you need to add three, or ten? You would need to rewrite the function or create multiple overloaded versions. Advanced function features solve this elegantly.

Problems with basic/rigid function signatures in production code:

- Cannot handle varying numbers of inputs — brittle and inflexible.
- Forces callers to remember exact parameter order — error-prone.
- Leads to code duplication — violates the DRY principle.
- Makes APIs and libraries hard to extend without breaking changes.

■ DATA ENGINEERING

In Data Engineering, functions are the building blocks of transformation logic inside ETL jobs. A function that can only handle a fixed schema will break every time the upstream data changes. Advanced function signatures make your code **schema-agnostic and resilient**.

3.2 Advanced Functions in Industry

Here is how these advanced features map to real industry scenarios:

Feature	Industry Use Case
*args	Aggregation functions that accept any number of column values or records
**kwargs	Dynamic configuration: passing DB connection params, Spark options, API headers
lambda	Inline transformations in map/filter, pandas apply(), PySpark UDFs
map()	Element-wise data transformation across large collections — batch ETL
filter()	Data quality filtering: removing nulls, outliers, flagged records
Clean Code	Maintainable pipelines readable by the whole team

4. Arbitrary Arguments

4.1 *args — Multiple Positional Arguments

***args** allows a function to accept **any number of positional arguments**. Internally, Python collects all the extra positional arguments into a **tuple**. The asterisk (*) is the syntax marker; *args* is just the conventional name (you could use **numbers*, **values*, etc.).

■ KEY CONCEPT

Syntax: `def function_name(*args):` — the * unpacks an arbitrary number of positional arguments into a tuple called *args*.

Example — Summing Any Number of Values

■ *args — Basic Addition

```
# *args collects all positional arguments into a tuple
def add(*values):
    total = 0
    for value in values:
        total += value
    return total

print(add(1, 2)) # Output: 3
print(add(4, 5, 6, 7)) # Output: 22
print(add(6, 66, 7, 8, 9)) # Output: 96
```

How It Works — Step by Step

- When you call `add(1, 2)`, Python collects (1, 2) into the tuple **values = (1, 2)**.
- The for loop iterates over the tuple, accumulating the sum.
- When you call `add(6, 66, 7, 8, 9)`, the same function works for 5 values — no changes needed.
- The parameter name *values* (or *args*) is just a naming convention — the * is what matters.

More Practical Examples

■ *args — Greeter & Logger

```
# Greet multiple people
def greet(*names):
    for name in names:
        print(f'Hello, {name}!')

greet('Alice', 'Bob', 'Charlie')
# Hello, Alice!
# Hello, Bob!
# Hello, Charlie!

# Mixed: fixed + *args
def log_event(event_type, *messages):
```

```
for msg in messages:
    print(f'[{event_type}] {msg}')

log_event('ERROR', 'DB timeout', 'Retry failed', 'Alert sent')
# [ERROR] DB timeout
# [ERROR] Retry failed
# [ERROR] Alert sent
```

■ NOTE

*args is internally a **tuple** — ordered, iterable, and immutable. You can iterate over it, index into it (args[0]), and use len(args) to check how many arguments were passed.

**■ DATA
ENGINE
ERING**

In Data Engineering: *args is used in aggregation UDFs (User Defined Functions) in PySpark or pandas where you want to apply the same logic across a variable number of column references. Example: a custom multi-column hasher, a dynamic GROUP BY aggregator.

4.2 **kwargs — Variable-length Keyword Arguments

****kwargs** (keyword arguments) allows a function to accept **any number of named/keyword arguments**. Python collects them into a **dictionary** where the keys are the argument names and the values are the argument values. The double asterisk (******) is the syntax marker.

**■ KEY
CONCEPT**

Syntax: def function_name(**kwargs): — kwargs becomes a dictionary of all named arguments passed to the function.

Example — Student Information

■ **kwargs — Student Info Printer

```
def student_info(**kwargs):
    for key, value in kwargs.items():
        print(key, value)
```

```
student_info(
    name = 'Tanishq',
    roll_no = 123,
    email = 'abc@gmail.com',
    mobile_number = 50989058,
    isIndian = True
)
```

Output:

name Tanishq

roll_no 123

email abc@gmail.com

mobile_number 50989058

isIndian True

Why kwargs Returns a Dictionary

Inside the function, **kwargs** behaves exactly like a Python dictionary. You can use all standard dict methods: `.items()`, `.keys()`, `.values()`, `.get()`, and `len(kwargs)`. This makes it extremely flexible for processing named configurations dynamically.

■ ****kwargs – Pipeline Config Printer**

```
def describe_pipeline(**config):
    print('Pipeline Configuration:')
    for param, val in config.items():
        print(f' {param}: {val}')
    print(f'Total params: {len(config)}')

describe_pipeline(
    source = 'S3',
    target = 'Redshift',
    schedule = 'daily',
    batch_size = 10000,
    retry = True
)
```

■ **DATA ENGINE ERING**

In Data Engineering: ****kwargs** is the backbone of **dynamic configuration**. When building generic connectors (e.g., a database writer that works with MySQL, Postgres, or BigQuery), you pass connection parameters as ****kwargs** so the same function works for all backends.

4.3 Combining *args and **kwargs

You can use both in the same function. When combined, ***args must always come before **kwargs**. Python enforces this rule — reversing the order causes a `SyntaxError`.

■ ***args + **kwargs Together**

```
def demo(*tuple_values, **dictionary_values):
    print('Positional args (tuple):', tuple_values)
    print('Keyword args (dict):', dictionary_values)

demo(1, 2, 3, 4, 5, name='Tanishq', last_name='Tyagi')

# Output:
# Positional args (tuple): (1, 2, 3, 4, 5)
# Keyword args (dict): {'name': 'Tanishq', 'last_name': 'Tyagi'}
```

■ **IMP ORTANT**

Order Rule: In a function signature, the order must always be: `def func(regular_params, *args, **kwargs)`. Placing ****kwargs** before ***args** raises a `SyntaxError`.

■ **Correct vs Incorrect Ordering**

```
# WRONG – this will cause a SyntaxError
# def demo_2(**kwargs, *args): <-- SyntaxError!
# print(args)
```

```
# print(kwargs)

# CORRECT order

def complete_func(name, age, *hobbies, **settings):
    print(f'{name}, {age}')
    print('Hobbies:', hobbies)
    print('Settings:', settings)
```

4.4 *args vs **kwargs — Key Differences

Aspect	*args	**kwargs
Full form	Arbitrary positional arguments	Arbitrary keyword arguments
Data structure	Tuple	Dictionary
How values are passed	Positionally: func(1, 2, 3)	By name: func(a=1, b=2)
Access method	Iteration, indexing	.items(), .keys(), .values()
Order enforced?	Yes — order matters	No — names identify values
Typical use case	Variable-count numeric ops	Dynamic config / named params
Syntax marker	Single asterisk (*)	Double asterisk (**)

5. Lambda Functions

A **lambda function** is a **small, anonymous (unnamed) function** defined in a single line. Unlike regular functions defined with **def**, lambda functions have no name and consist of a **single expression** whose result is automatically returned. They are often called *throwaway functions* — defined once and used inline.

■ KEY CONCEPT

Syntax: `lambda arguments : expression`

- **lambda** — keyword that signals an anonymous function
- **arguments** — one or more comma-separated inputs (like regular function params)
- **expression** — a single expression whose value is returned automatically

5.1 Single Argument Lambda

■ Lambda — Single Argument

```
# Traditional function
def square(x):
    return x * x
print(square(4)) # Output: 16

# Equivalent lambda — called immediately
print((lambda x: x * x)(4)) # Output: 16

# Assigned to a variable (common pattern)
square = lambda x: x * x
print(square(5)) # Output: 25
print(square(9)) # Output: 81
```

5.2 Multiple Arguments Lambda

■ Lambda — Multiple Arguments

```
# Lambda with two arguments
calculate = lambda a, b: a + b
print(calculate(2, 3)) # Output: 5

# Lambda for power
power = lambda base, exp: base ** exp
print(power(2, 10)) # Output: 1024

# Lambda with three arguments
full_name = lambda first, middle, last: f'{first} {middle} {last}'
print(full_name('Tanishq', 'Kumar', 'Tyagi'))
# Output: Tanishq Kumar Tyagi
```

5.3 When to Use Lambdas

- Use **lambda** for short, one-time-use functions — especially as arguments to `map()`, `filter()`, `sorted()`.

- **Use def** for functions with multiple statements, complex logic, or docstrings.
- Lambda functions improve code readability for simple inline operations.
- Avoid using lambda for logic that needs to be tested, debugged, or reused.

**■■ IMP
ORTANT**

Common Mistake: Assigning lambdas to variables defeats the purpose. `square = lambda x: x*x` is legal Python but PEP8 advises using `def square(x): return x*x` instead — it's more readable and debuggable.

**■ DATA
ENGINE
ERING**

In Data Engineering: Lambdas are used extensively with **pandas .apply()**, **PySpark .withColumn()** / UDFs, and **map()/filter()** in data transformation pipelines. Example:
`df['salary_usd'] = df['salary_inr'].apply(lambda x: x / 83.5)`

6. The map() Function

map() is a built-in Python higher-order function that applies a **function to every element** in an iterable (list, tuple, set, etc.) and returns a **map object** (a lazy iterator). You must convert it to a list or tuple to view the results. **map()** does not modify the original iterable.

■ KEY CONCEPT

Syntax: `map(function, iterable)`
Flow: Each item from *iterable* → passed to *function* → result collected.
Returns a map object (lazy) — wrap with `list()` or `tuple()` to materialise results.

6.1 Syntax & Flow

■ map() — Basic Usage

```
# Flow: item from list -> function -> result
```

```
marks_list = [90, 85, 80, 50]
```

```
# Regular function approach
```

```
def add_5(num):
```

```
    return num + 5
```

```
updated_marks = map(add_5, marks_list)
```

```
print(list(updated_marks)) # [95, 90, 85, 55]
```

```
# Same with lambda — more concise
```

```
updated_marks = map(lambda x: x + 5, marks_list)
```

```
print(list(updated_marks)) # [95, 90, 85, 55]
```

■■ IMPORTANT

Important: A map object is a **lazy iterator** — it is consumed on first use. After you call `list(updated_marks)` once, the iterator is exhausted. Calling `tuple(updated_marks)` immediately after will return an **empty tuple**. Always store the materialised result: `result = list(map(...))`.

6.2 Practical Examples

■ map() — Practical Transformations

```
# Convert names to uppercase
```

```
names_list = ['tanishq', 'ayush', 'moksh']
```

```
upper_names = list(map(str.upper, names_list))
```

```
print(upper_names) # ['TANISHQ', 'AYUSH', 'MOKSH']
```

```
# Square every number
```

```
nums = [1, 2, 3, 4, 5]
```

```
squares = list(map(lambda x: x ** 2, nums))
```

```
print(squares) # [1, 4, 9, 16, 25]
```

```
# Convert salary from INR to USD (exchange rate: 83.5)
```

```
salaries_inr = [50000, 100000, 75000, 120000]
salaries_usd = list(map(lambda s: round(s / 83.5, 2), salaries_inr))
print(salaries_usd) # [598.8, 1197.6, 898.2, 1437.13]
```

■ DATA ENGINE ERING

In Data Engineering: **map()** is the conceptual basis for distributed **MapReduce** — the paradigm behind Apache Hadoop and Spark. In Python pipelines, **map()** is ideal for **element-wise transformations** on lists of records: currency conversion, unit normalization, encoding IDs, standardising date formats, etc.

7. The filter() Function

filter() is a built-in Python higher-order function that **tests every element** in an iterable against a **predicate function** (a function that returns True or False). Elements for which the function returns **True** are **kept**; elements returning **False** are **removed**. Like `map()`, it returns a **lazy iterator**.

■ KEY CONCEPT

Syntax: `filter(function, iterable)`

Rule: function returns True → Keep element | function returns False → Remove element
Returns a filter object (lazy) — wrap with `list()` to materialise.

7.1 Syntax & Flow

■ filter() – Basic Usage

```
marks_list = [90, 85, 80, 50]

# Keep only marks >= 80
def is_gte_80(num):
    return num >= 80

filtered = filter(is_gte_80, marks_list)
print(list(filtered)) # [90, 85, 80]

# Same with lambda
filtered = filter(lambda num: num >= 80, marks_list)
print(list(filtered)) # [90, 85, 80]
```

7.2 Practical Examples

■ filter() – Practical Filtering

```
# Filter even numbers
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens) # [2, 4, 6, 8, 10]

# Filter salaries above 50,000
salaries = [30000, 55000, 45000, 72000, 68000, 25000]
high_salaries = list(filter(lambda s: s > 50000, salaries))
print(high_salaries) # [55000, 72000, 68000]

# Filter non-empty strings
data = ['Alice', '', 'Bob', None, 'Charlie', '']
clean = list(filter(None, data)) # None as function removes falsy values
print(clean) # ['Alice', 'Bob', 'Charlie']
```

map() vs filter() — Quick Comparison

Aspect	map()	filter()
--------	-------	----------

Purpose	Transform every element	Selectively keep elements
Function returns	Transformed value	Boolean (True/False)
Output length	Same as input	$\leq$ input length
Typical use	Data transformation	Data quality / selection
Example	Add 5 to each mark	Keep marks ≥ 80

**■ DATA
ENGINE
ERING**

In Data Engineering: **filter()** is used for **data quality checks** — removing null rows, flagged records, out-of-range values, duplicate entries. In PySpark the equivalent is `.filter()` or `.where()` on a DataFrame.

8. Clean Code Principles

Clean Code is code that is easy to **read, understand, maintain, and extend**. The concept was popularised by Robert C. Martin ("Uncle Bob") and is a fundamental professional standard in all software engineering disciplines — including Data Engineering. Clean code is not just about correctness; it is about **communicating intent** clearly to the next developer (who is often yourself six months later).

8.1 Meaningful Names

Every variable, function, and class should have a name that **clearly reveals its intent**. If a name requires a comment to explain it, the name is not good enough.

■ Meaningful Names

```
# BAD – cryptic names
def calc(x, y):
    return x * y / 100

# GOOD – self-documenting names
def calculate_tax(income, tax_rate_percent):
    return income * tax_rate_percent / 100

# BAD variable names
d = 30
t = 10

# GOOD variable names
days_in_month = 30
temperature_celsius = 10
```

8.2 Small Functions

Functions should **do one thing** and do it well. As a rule of thumb, a function should be short enough to fit on one screen (roughly 10–20 lines). Long functions are hard to test, debug, and reuse.

■ Small Functions

```
# BAD – one giant function doing everything
def process_student_data(students):
    # validate, calculate average, filter, format, print – all in one
    ...

# GOOD – split into small, focused functions
def calculate_average(marks):
    return sum(marks) / len(marks)

def filter_passing_students(students, pass_mark=40):
    return [s for s in students if s['average'] >= pass_mark]

def format_student_report(student):
    return f'{student["name"]}: {student["average"]:.1f}'
```

8.3 Avoid Repetition (DRY Principle)

DRY = Don't Repeat Yourself. If you find yourself copying and pasting the same block of code, that code should become a function. Every piece of knowledge should have a **single, unambiguous representation** in your codebase.

■ DRY Principle

```
# BAD – repetition
marks_a = [90, 85, 75]
avg_a = sum(marks_a) / len(marks_a)

marks_b = [70, 65, 80]
avg_b = sum(marks_b) / len(marks_b)

# GOOD – DRY
def find_average(marks):
    return sum(marks) / len(marks)

avg_a = find_average(marks_a)
avg_b = find_average(marks_b)
```

8.4 Single Responsibility Principle (SRP)

A function (or class) should have **one reason to change** — meaning it should do exactly one thing. If a function both calculates a result AND prints it AND saves it to a file, it has three responsibilities and will be fragile to changes in any one of them.

■ Single Responsibility Principle

```
# BAD – too many responsibilities
def process_and_print_and_save(data):
    result = sum(data) / len(data)
    print(f'Average: {result}')
    with open('output.txt', 'w') as f:
        f.write(str(result))

# GOOD – each function does one thing
def calculate_average(data):
    return sum(data) / len(data)

def display_result(result):
    print(f'Average: {result}')

def save_result(result, filename):
    with open(filename, 'w') as f:
        f.write(str(result))
```

■ DATA ENGINEERING

In Data Engineering: Clean Code is critical because data pipelines are maintained by teams. A pipeline with cryptic function names and 200-line functions becomes impossible to debug at 2AM during a production incident. SRP and DRY are foundational to writing **testable, observable, and maintainable ETL jobs**.

9. Practical Examples

The following examples combine multiple concepts from this session into cohesive, real-world-style programs.

Example 1 — Data Pipeline Transformation

■ Complete Pipeline Example

```
# Combining *args, lambda, map, filter, and clean code

def calculate_average(*marks):
    """Accepts any number of marks and returns their average."""
    return sum(marks) / len(marks) if marks else 0

student_scores = [45, 78, 92, 33, 88, 61, 55, 71]

# Step 1: Add grace marks using map
GRACE = 5
scores_with_grace = list(map(lambda s: s + GRACE, student_scores))
print('After grace:', scores_with_grace)
# [50, 83, 97, 38, 93, 66, 60, 76]

# Step 2: Filter passing students (>= 40)
PASS_MARK = 40
passing_scores = list(filter(lambda s: s >= PASS_MARK, scores_with_grace))
print('Passing scores:', passing_scores)
# [50, 83, 97, 38 removed, 93, 66, 60, 76]

# Step 3: Calculate average
avg = calculate_average(*passing_scores)
print(f'Class average (passing students): {avg:.2f}')
```

Example 2 — Dynamic Config with **kwargs

■ **kwargs for Dynamic DB Config

```
def create_db_connection(**config):
    """Creates a connection string from any DB config params."""
    host = config.get('host', 'localhost')
    port = config.get('port', 5432)
    db = config.get('database', 'default_db')
    user = config.get('user', 'admin')
    return f'postgresql://{user}@{host}:{port}/{db}'

conn_str = create_db_connection(
    host='prod-db.company.com',
    port=5432,
    database='data_warehouse',
```

```
user='etl_user'  
)  
print(conn_str)  
# postgresql://etl_user@prod-db.company.com:5432/data_warehouse
```

10. Real-World Data Engineering Perspective

■ DATA ENGINE ERING

***args in ETL**

*args is used to write generic aggregation and transformation functions that work on a variable number of columns or records. In Apache Spark (PySpark), custom aggregation UDFs often accept *args to handle dynamic column sets. In batch processing jobs, *args lets you write one function that handles daily, weekly, or monthly data sizes without modification.

■ DATA ENGINE ERING

****kwargs for Dynamic Configuration**

Production data pipelines are highly configurable. Database connectors, API wrappers, and cloud storage writers all accept **kwargs to handle environment-specific parameters (dev vs. staging vs. prod). Libraries like SQLAlchemy, boto3 (AWS), and google-cloud-bigquery all expose **kwargs-style interfaces for their connection and query methods.

■ DATA ENGINE ERING

Lambda in Data Transformations

Lambda functions are used extensively in pandas .apply(), PySpark .withColumn(), and Python's built-in map/filter within pipeline stages. They keep transformation logic inline and readable without polluting the global namespace with single-use named functions. AWS Lambda (the cloud service) is named after this concept — small, single-purpose compute units.

■ DATA ENGINE ERING

map() — The M in MapReduce

The map() concept is foundational to the MapReduce programming model used by Apache Hadoop and Spark. In Python pipelines, map() is used for element-wise transformations: normalising values, applying exchange rates, encoding categorical features, or applying regex patterns to strings — all without writing explicit for loops.

■ DATA ENGINE ERING

filter() for Data Quality

Data quality is a primary concern in every production pipeline. filter() and its equivalents (DataFrame.filter() in PySpark, DataFrame.query() in pandas) are used to remove null records, filter out rows below SLA thresholds, exclude test/synthetic data, and enforce business rules (e.g. only process transactions > 0).

■ DATA ENGINE ERING

Clean Code in Production Pipelines

In production, pipelines are owned by teams and must be maintained for years. Clean code practices — meaningful names, small single-purpose functions, no repetition — directly reduce time-to-debug during incidents, improve onboarding time for new engineers, enable unit testing of individual transformation steps, and make pipeline DAGs (Directed Acyclic Graphs in Airflow/Prefect) self-documenting.

11. Summary & Quick Revision Sheet

Key Concepts Summary

Concept	What It Is	Key Rule
*args	Collects extra positional args into a tuple	Iterate with for loop; use len(args)
**kwargs	Collects extra keyword args into a dict	Use .items(), .get(); key=value syntax
lambda	Anonymous single-expression function	lambda args : expression; returns auto
map()	Apply function to every element	Returns lazy iterator; wrap with list()
filter()	Keep elements where function is True	Returns lazy iterator; wrap with list()
Meaningful names	Names reveal intent	No abbreviations; no single letters (except loops)
Small functions	One function = one task	~10-20 lines max; testable in isolation
DRY	Don't Repeat Yourself	If copy-pasting, extract a function
SRP	Single Responsibility Principle	One reason to change; one purpose per function

Syntax Cheat Sheet

■ Complete Syntax Reference

```
# *args
def func(*args): # args is a tuple
    for item in args: ...

# **kwargs
def func(**kwargs): # kwargs is a dict
    for k, v in kwargs.items(): ...

# Both together (order matters!)
def func(*args, **kwargs): ...

# Lambda
f = lambda x: x * 2
f = lambda x, y: x + y

# map
result = list(map(lambda x: x+1, my_list))
result = list(map(function_name, my_list))

# filter
result = list(filter(lambda x: x > 0, my_list))
result = list(filter(predicate_function, my_list))
```

Common Mistakes & Reminders

- **map() / filter() are lazy** — always wrap with list() or tuple() before printing.
- ***args before **kwargs** — reversing the order in a function signature causes `SyntaxError`.
- **Lambda has one expression** — cannot contain statements like if/else blocks, for loops, or assignments.
- **Consumed iterators** — calling list() twice on the same map/filter object gives empty result the second time.
- **Don't over-lambda** — if logic is complex, use a named def function for readability.
- **args/kwargs are conventions** — *data, **config are equally valid names; the * and ** are what matter.

12. General Python Interview Questions

A. Beginner Level

Q: What is `*args` in Python?

A: `*args` allows a function to accept any number of positional arguments. Python collects them into a tuple inside the function. The asterisk (`*`) is the syntax marker, and `'args'` is a naming convention — you can use any name like `*values` or `*numbers`.

Q: What is `**kwargs` in Python?

A: `**kwargs` allows a function to accept any number of keyword (named) arguments. Python collects them into a dictionary inside the function. The double asterisk (`**`) is the syntax marker.

Q: What is a lambda function?

A: A lambda function is a small anonymous (nameless) function defined using the lambda keyword. It can have any number of arguments but only one expression. Syntax: `lambda args : expression`. It automatically returns the value of the expression.

Q: What does the `map()` function do?

A: `map()` applies a function to every element of an iterable and returns a lazy map object. You must convert it to a list or tuple to see the results. Syntax: `map(function, iterable)`.

Q: What does `filter()` return?

A: `filter()` returns a lazy filter object containing only the elements for which the given function returned `True`. Like `map()`, it must be converted to `list()` or `tuple()` to materialise the results.

Q: What is the difference between `map()` and `filter()`?

A: `map()` transforms every element (output length = input length). `filter()` selects elements based on a condition (output length $\leq$ input length). `map()` applies a transformation; `filter()` applies a test.

Q: Can you use `*args` and `**kwargs` together?

A: Yes. When used together, `*args` must always come before `**kwargs` in the function signature. Example: `def func(*args, **kwargs)`. Positional values go to `args` (tuple) and named values go to `kwargs` (dict).

Q: What is the DRY principle?

A: DRY stands for Don't Repeat Yourself. It means every piece of logic should exist in only one place in the codebase. If you find yourself copy-pasting code, that code should become a reusable function.

Q: What data structure does `*args` produce inside a function?

A: A tuple. It is ordered, iterable, and immutable. You can use `len(args)`, iterate over it with `for`, or index into it with `args[0]`.

Q: What data structure does `**kwargs` produce inside a function?

A: A dictionary. Keys are the argument names (strings) and values are the argument values. You can use `.items()`, `.keys()`, `.values()`, `.get()`, and `len()` on it.

Q: What is an anonymous function?

A: An anonymous function is a function without a name. In Python, lambda functions are anonymous. They are defined with the lambda keyword and used for short, single-use operations — typically passed as arguments to `map()`, `filter()`, or `sorted()`.

B. Intermediate Level

Q: What happens if you call `list()` on an already-consumed map object?

A: You get an empty list. `map()` and `filter()` return lazy iterators — they are consumed once. After calling `list(m)` the first time, the iterator is exhausted. Always store the result: `result = list(map(...))`.

Q: What is the Single Responsibility Principle?

A: SRP states that a function or class should have only one reason to change — i.e., one purpose. If a function calculates a result AND prints it AND saves it, it has three responsibilities and becomes fragile to change. Split it into three focused functions.

Q: Can a lambda function contain if-else logic?

A: Yes, but only as a conditional expression (ternary operator): `lambda x: x if x > 0 else -x`. It cannot contain multi-line if/else blocks, for loops, or assignment statements — those require a def function.

Q: What is the difference between a named function and a lambda function?

A: A named function (`def`) can have multiple statements, docstrings, can be tested and debugged by name, and is more readable for complex logic. A lambda is anonymous, limited to one expression, and best suited for short inline operations passed to other functions.

Q: How do you pass `*args` to another function?

A: Using the unpacking operator: `def outer(*args): return inner(*args)`. The `*` unpacks the tuple back into positional arguments for the inner function.

Q: What are higher-order functions?

A: Higher-order functions are functions that accept other functions as arguments or return functions as results. `map()`, `filter()`, and `sorted()` are examples. They enable functional programming patterns — clean, declarative data transformations.

Q: When should you prefer list comprehension over `map()`/`filter()`?

A: For simple transformations on small-to-medium datasets, list comprehensions are generally more Pythonic and readable: `[x*2 for x in data]` vs `list(map(lambda x: x*2, data))`. `map()` can be faster for very large datasets and avoids creating an intermediate list.

Q: Can you use `map()` with multiple iterables?

A: Yes. `map()` can accept multiple iterables: `map(lambda x, y: x+y, list1, list2)`. The function receives one element from each iterable per iteration. The output length matches the shortest input iterable.

Q: What does `filter(None, iterable)` do?

A: When `None` is passed as the function, `filter()` removes all falsy values from the iterable: `False`, `0`, `None`, empty strings `"`, and empty lists `[]`. This is a quick way to clean a list of null-like values.

Q: What is the order rule for `*args` and `**kwargs`?

A: The correct order in a function signature is: regular parameters first, then `*args`, then keyword-only parameters, then `**kwargs`. Example: `def func(a, b, *args, key=default, **kwargs)`. Violating this order causes a `SyntaxError`.

C. Advanced Level

Q: What is the difference between `map()` and a list comprehension in terms of memory?

A: `map()` returns a lazy iterator — it generates values on demand and uses $O(1)$ memory regardless of input size. A list comprehension materialises all results into memory at once — $O(n)$ memory. For very large datasets (millions of rows), chained `map()` calls can be significantly more memory efficient.

Q: Can you pass a class method or a built-in function to `map()`?

A: Yes. `map()` accepts any callable: lambda, def functions, built-in functions, class methods, and static methods. Example: `list(map(str.upper, names))` uses the built-in string method directly without a lambda

wrapper.

Q: How does `**kwargs` relate to dictionary unpacking?

A: They share the same `**` operator. In function definitions, `**kwargs` collects keyword arguments into a dict. In function calls, `**dict_name` unpacks a dictionary into keyword arguments. This symmetry allows: `config = {'host': 'db.co', 'port': 5432}; connect(**config)` — very powerful for passing dynamic configs.

Q: What is a closure, and how do lambdas relate to it?

A: A closure is a function that captures variables from its enclosing scope. Lambdas can form closures: `def multiplier(n): return lambda x: x * n; double = multiplier(2); triple = multiplier(3)`. Here, `n` is captured from the outer scope. This is the basis for function factories in Python.

Q: How would you implement a pipeline of transformations using `map()` and `filter()`?

A: You chain them: `result = list(filter(predicate, map(transform, data)))`. For multiple steps: `pipeline = data; for step in [map_fn, filter_fn]: pipeline = step(pipeline)`. This pattern mirrors functional programming and is how pandas/Spark method chains work internally.

13. Data Engineer Interview Preparation

The following questions specifically target how **Advanced Functions** concepts appear in real Data Engineering roles — covering ETL pipelines, PySpark, Airflow, and production systems.

DE-Q1: How do you use `*args` in a PySpark UDF?

A: In PySpark, you register UDFs with a fixed schema, but you can pass `*args` to the underlying Python function to handle a variable number of column references dynamically. For example, a custom hasher that combines N columns: `def hash_columns(*cols): return hashlib.md5(''.join(map(str, cols)).encode()).hexdigest()`. You'd then wrap this as a UDF and apply it. This avoids writing separate functions for 2-column, 3-column, or 5-column hash keys.

DE-Q2: Describe a scenario where `**kwargs` was useful in a production data pipeline.

A: When building a generic database writer that needs to support MySQL, PostgreSQL, and BigQuery, you define: `def write_to_db(table_name, data, **conn_params)`. The `conn_params` dict contains driver-specific settings (SSL certificates for Postgres, `project_id` for BigQuery, `charset` for MySQL). The caller passes only the relevant params. This pattern avoids a bloated function signature and makes the writer extensible without code changes.

DE-Q3: How does the `map()` function concept relate to distributed computing in Spark?

A: Spark's core programming model is MapReduce — the Map phase applies a transformation function to each partition/record independently (embarrassingly parallel), and the Reduce phase aggregates results. Python's `map()` is the single-machine analogue. When you call `rdd.map(transform_fn)` in PySpark, each executor applies `transform_fn` to its partition. Understanding Python's `map()` makes the Spark mental model intuitive.

DE-Q4: When would you use `filter()` in an ETL pipeline?

A: `filter()` is used in ETL for data quality enforcement: (1) removing null or malformed records before loading to the warehouse, (2) filtering out test/synthetic data rows based on a flag column, (3) enforcing SLA windows (only process events within the last 24 hours), (4) removing duplicate entries before upsert, (5) applying business rule exclusions (skip cancelled orders, exclude negative amounts).

DE-Q5: How would you write a clean, production-grade ETL function following Clean Code principles?

A: A production ETL function should: (1) have a single responsibility — extract, transform, or load, not all three, (2) use meaningful names like `extract_customer_events()` not `process_data()`, (3) accept configuration via `**kwargs` or a typed config object, (4) not print — use a logger, (5) return transformed data rather than side effects, (6) be independently testable with a unit test, (7) have a docstring explaining inputs, outputs, and side effects.

DE-Q6: What is a lambda function in the context of AWS, and how does it differ from Python lambda?

A: AWS Lambda is a serverless compute service — you deploy a function without managing servers, and it runs in response to events. It is named after the lambda calculus concept (anonymous, stateless functions). Python's `lambda` keyword creates anonymous single-expression functions. The shared name reflects the same philosophy: small, single-purpose, stateless compute units. In data pipelines, AWS Lambda functions often contain Python lambda expressions for data transformations.

DE-Q7: How do you handle schema evolution using `**kwargs` in a pipeline?

A: Schema evolution (new columns added upstream) is a common challenge. By accepting `**record_fields` as kwargs, a transformation function can handle new fields gracefully: it processes only the fields it knows

about and ignores the rest. This prevents pipelines from breaking when the upstream schema adds a column. Combined with `.get()` defaults, this is a resilient pattern: `name = record_fields.get('customer_name', 'UNKNOWN')`.

DE-Q8: What is the performance difference between `map()` and a `for` loop in Python for large datasets?

A: `map()` is marginally faster than an equivalent `for` loop because the iteration happens in C (CPython internals), while a `for` loop iterates in Python bytecode. However, the more significant advantage is memory: `map()` is a lazy iterator and processes one element at a time, while a list comprehension or `for` loop building a list allocates memory for all results. For multi-million-row datasets, lazy `map()` chains prevent OOM (Out of Memory) errors.

DE-Q9: How would you use `filter()` and `map()` together in a data quality pipeline?

A: A typical pattern: Step 1 — use `filter()` to remove invalid records (nulls, out-of-range values, malformed strings). Step 2 — use `map()` to transform the remaining valid records (normalisation, encoding, enrichment). Chaining: `clean_data = list(map(transform, filter(is_valid, raw_data)))`. This separates validation logic from transformation logic — each function has a single responsibility.

DE-Q10: Describe the role of lambda functions in pandas ETL workflows.

A: In pandas, `.apply()` is the primary way to use lambda: `df['col'] = df['col'].apply(lambda x: transform(x))`. Lambdas handle inline transformations: currency conversion, string normalisation, conditional flags, type casting. For column-wise operations, they're passed to `.map()`, `.applymap()`, or `.transform()`. The key advantage is keeping transformation logic inline and readable without defining a separate named function for every minor step.

DE-Q11: What is the risk of using overly complex lambda functions in production?

A: Complex lambdas are undebuggable by name (they show as in tracebacks), untestable in isolation, and unreadable to other engineers. In production, a lambda like: `lambda x: x['a'] if x.get('b') and not x.get('c') else x.get('d', 0)` should immediately be refactored into a named `def` function with a docstring. The rule: if your lambda needs more than one line to read, use `def`.

DE-Q12: How do `*args` and `**kwargs` support the design of reusable pipeline components?

A: Reusable components must be flexible by design. `*args` allows a transformer to work on any number of columns or records. `**kwargs` allows passing any configuration options (`source_format`, `delimiter`, `encoding`, `null_value`) without changing the function signature. Combined, they produce components that behave correctly regardless of the specific pipeline they're embedded in — following the Open/Closed Principle: open for extension, closed for modification.

DE-Q13: What is the relationship between Clean Code and pipeline observability?

A: Observability (monitoring, alerting, debugging) depends on code that is structured around single-responsibility functions with meaningful names. When a pipeline fails, a stack trace through `process_raw_data()` → `validate_schema()` → `transform_timestamps()` → `load_to_warehouse()` immediately tells you which stage failed. A stack trace through `process()` → `process2()` → `do_stuff()` tells you nothing. Clean Code is a prerequisite for effective observability.

DE-Q14: How would you implement a configurable transformation pipeline using `*args` and `**kwargs`?

A: Design: `def run_pipeline(*transformations, **config): data = config.get('source_data', []); for transform in transformations: data = transform(data, **config); return data`. Then call: `run_pipeline(clean_nulls, normalise_dates, encode_categoricals, source_data=raw_df, date_format='%Y-%m-%d', encoding='label')`. Each transform function receives the shared config via `**config`, making the pipeline fully configurable without hardcoded parameters.

DE-Q15: What are the performance implications of using `filter()` vs `DataFrame.filter()` in PySpark?

A: Python's built-in `filter()` operates on Python objects in a single-threaded, single-machine context. PySpark's `DataFrame.filter()` (or `.where()`) is translated into a Catalyst query plan and executed across distributed executors with predicate pushdown — the filter is pushed as close to the data source as possible (e.g., reading only matching rows from S3/Parquet). For any dataset larger than memory, always use PySpark's native `.filter()` rather than converting to Python and using the built-in `filter()`.